

Module 1 — Core Mastery

Lesson 1.4

Plan mode

A read-only mode where Claude designs the change before it touches anything. Use it on every non-trivial task.

Mastering Claude Code — An E-Course for Power Users

Why it matters

Most bugs from agentic coding come from Claude jumping to implementation before it understands the problem. Plan mode separates research from action: Claude explores, asks clarifying questions, writes a plan to a file, and only proceeds after you approve.

For anything bigger than a one-liner edit, plan mode is the cheapest way to avoid waste.

The mental model

Plan mode active:



The plan lives in a markdown file in your `~/.claude/plans/` directory; Claude is told its exact path on entry to plan mode.

How to enter

Three ways:

- 1 **Launch flag:** `claude --permission-mode plan`
- 2 **Mid-session toggle:** Shift+Tab cycles through permission modes (default → acceptEdits → plan → back)
- 3 **User instruction:** just ask — "Let's plan this before you implement."

The plan workflow (in order)

- 1 **Understand** — Claude reads relevant files, optionally launches Explore subagents to map unfamiliar terrain.
- 2 **Design** — Claude considers approaches, often launching a Plan subagent for a second perspective.
- 3 **Clarify** — Claude asks you any open questions (AskUserQuestion).
- 4 **Write** — Claude writes the final plan to its plan file.
- 5 **ExitPlanMode** — Claude calls this tool to signal "ready for your review."
- 6 **Approve or revise** — You read the plan, accept or send corrections.

What a good plan contains

- **Context** — why this change, what problem it solves

- **Approach** — the recommended implementation (not all alternatives)
- **Files to touch** — explicit paths
- **Reused utilities** — what already exists that shouldn't be re-built
- **Verification** — how you'll know it worked (tests, manual steps, success criteria)

A bad plan is a vague essay. A good plan is something a junior on your team could execute.

When to use it

Task	Plan mode?
Fix a typo	No
Rename one variable	No
Add a new field to an API response	Yes
Refactor a module	Yes
Debug a flaky test	Yes (research first)
One-shot exploration ("how does X work?")	Sometimes — it locks you read-only, which is fine

Rule of thumb: if the change touches more than one file, plan first.

Try it

- 1 In any repo, launch `claude --permission-mode plan`.
- 2 Ask: "I want to add a `/healthz` endpoint that returns 200 if the database is reachable. Plan it."
- 3 Watch the Explore/Plan flow.
- 4 Read the resulting plan file. Push back on any vague step.
- 5 Approve and let Claude implement.

Gotchas

- **Don't ask Claude to "use ExitPlanMode" while it's still researching.** It will refuse — that tool is for after the plan file is written.
- **Plan mode persists.** If you Shift+Tab into it and forget, edits in your next prompt will fail. Watch the mode indicator.
- **The plan file is Claude's working doc, not your team's design doc.** Don't expect a polished spec — expect a punch list.
- **``AskUserQuestion`` inside plan mode is for requirements clarification only.** Not for "should I exit plan mode now?" — that's what ExitPlanMode is for.

Further reading

- 2.1 The planning loop — applying plan mode in a real loop
- 4.3 Prompting subagents — Plan subagent for second opinions

Next

→ 1.5 Context management